

PySpark on Databricks ハンズオン資料

PySpark の概念とアーキテクチャから、DataFrame 操作・SQL 連携・実践的なデータエンジニアリング設計・パフォーマンスチューニングまで。初級から中級までを1つにまとめた実践資料です。

 Part 1 — 初級：基礎

 Part 2 — 中級：実践

Part 1 — 初級：PySpark の基礎

Spark の考え方・アーキテクチャを押さえ、DataFrame の作成と基本操作、データソースの読み書きまでを学びます。

1. PySpark / Apache Spark の概要

Apache Spark は OSS の並列分散処理フレームワークです。障害時のリカバリ、タスク分割、スケジューリングを Spark が肩代わりし、サーバーを増やす（スケールアウト）ことでスループットがほぼ線形に向上します。**PySpark** はその Spark を Python から扱う API で、Python の手軽さと分散処理を両立できます。Databricks は Apache Spark の上に構築されたプラットフォームで、ノートブックには `spark`（SparkSession）と `dbutils` が初期化済みです。

高速

オンメモリ処理 / Project Tungsten / キャッシュ / 遅延評価による最適化。

多言語・多機能

Python・Scala・SQL から利用可。SQL / Streaming / MLlib / GraphX を内包。

スケーラブル

1 台でも動き、数千台のクラスターまで同じコードでスケール。

豊富なデータソース

CSV / JSON / Parquet / Delta / クラウドストレージ / DB に対応。

2 つのプログラミングモデル：RDD と DataFrame

	RDD（Spark Core）	DataFrame（Spark SQL）
記述スタイル	コレクション操作のように記述	SQL ライク・構造化データ処理
データ	非構造化データに柔軟（行指向）	スキーマを持つ表形式（列指向）
最適化	手動	オプティマイザ（Catalyst）が自動最適化
速度	Python では遅い（後述）	JVM 実行で高速

推奨： Spark の機能は徐々に DataFrame ベースへ移行しています。特別な理由がなければ **DataFrame を使う** のが基本です。

2. PySpark のアーキテクチャ（なぜ DataFrame が速いのか）

PySpark は Python と JVM を **Py4J** で橋渡しします。ここを理解すると、パフォーマンスの勘所が見えてきます。

Driver (Python)

SparkContext
(Python)

Py4J / socket

Executor (Worker)

JVM Task ← DataFrame はここで実行 (高速) |
Python proc ← RDD / Python UDF はここ (遅い) |

- **DataFrame** の処理はワーカー上の **JVM** で実行されるため高速。
- **RDD** や **Python UDF** はワーカー上の **Python プロセス** で実行され、JVM↔Python 間で シリアライズ (pickling) とパイプ転送のオーバーヘッドが発生します。Python Worker のメモリは JVM の制御外で、クラスター環境ではコンテナが kill されやすい点にも注意。

結論： できるだけ **DataFrame** と組み込み関数 (`pyspark.sql.functions`) で書く。Python UDF は最後の手段。

3. 遅延評価：変換とアクション

PySpark の最重要概念です。**変換（Transformation）** を積み重ねても計算は実行されず、**アクション（Action）** で初めて実行されます。これにより Spark は処理全体を見渡して最適化できます。

種別	例	挙動
変換	<code>select</code> / <code>filter</code> / <code>withColumn</code> / <code>join</code> / <code>groupBy</code>	新しい DataFrame を返すだけ。計算はされない
アクション	<code>count</code> / <code>collect</code> / <code>show</code> / <code>display</code> / <code>write</code>	ここで実行計画が走り、計算される

```
from pyspark.sql import functions as F
df = (spark.range(1, 1000)
      .withColumnRenamed("id", "n")
      .filter("n % 2 = 0")           # 変換：まだ計算されない
      .withColumn("squared", F.col("n") ** 2))

df.explain(mode="formatted")  # 実行計画だけを確認できる
df.count()                   # ← アクション。ここで初めて計算
```

4. DataFrame の作成と SparkSession

SparkSession (spark)

すべての入り口は `SparkSession`。Databricks では `spark` として用意済みです。スクリプト実行や共通関数内では明示的に取得します。

```
from pyspark.sql import SparkSession
spark = SparkSession.builder.getOrCreate()      # 外部スクリプト
spark = SparkSession.getActiveSession()         # 関数内で既存セッションを再利用

# 実行時設定は spark.conf
spark.conf.set("spark.sql.shuffle.partitions", "200")
spark.conf.get("spark.sql.shuffle.partitions")
```

注意： `SparkSession` はプロセスごとに独立が必要。 `multiprocessing` などでも共有すると問題が起きます。

DataFrame を作る 4 つの方法

スキーマは `"col1 type, col2 type"` の DDL 文字列で簡潔に指定できます。

```
import datetime
from pyspark.sql import Row
schema = "str_col string, int_col integer, date_col date"

# 1) 辞書のリスト
spark.createDataFrame([{"str_col": "abc", "int_col": 123, "date_col": datetime.date(2020, 1, 1)}], schema)
# 2) 多次元リスト / タプル
spark.createDataFrame([["abc", 123, datetime.date(2020, 1, 1)]], schema)
# 3) Row
spark.createDataFrame([Row(str_col="abc", int_col=123, date_col=datetime.date(2020, 1, 1))], schema)
# 4) 値を直接指定 (型は推論)
spark.createDataFrame([("Mikhail", 15), ("Zaky", 13)], ["name", "age"])

# テーブル / CSV から
spark.table("samples.tpch.customer")
spark.read.format("csv").option("header", True).option("inferSchema", True).load(path)
```

メタデータ (スキーマ) の確認

```
df.printSchema()          # ツリー表示
df.columns                 # 列名リスト
df.dtypes                  # (列名, 型) のタプルリスト
df.schema.simpleString()   # DDL 風の文字列
df.schema.jsonValue()["fields"] # JSON 形式の詳細
```

5. 列・行の基本操作

```
from pyspark.sql.functions import col
# --- 列 ---
df.select(col("c_custkey"), col("c_acctbal"))
df.withColumn("balance_flag", col("c_acctbal") > 1000) # 列の追加
df.withColumn("c_custkey", col("c_custkey").cast("string")) # 型変換
df.withColumnRenamed("old", "new")                       # リネーム
# --- 行 ---
df.filter((col("c_nationkey") == 20) & (col("c_acctbal") > 1000)) # 複数条件は括弧
df.distinct()                                                  # 重複削除
df.na.drop(); df.na.fill("0", subset=["c_acctbal"])            # NULL 処理
df.sort(col("c_acctbal").desc()).limit(10)                     # ソート+上位N
```

データ型ごとの操作イディオム

条件分岐・NULL・日付・複雑な型の扱い（『The Data Engineer's Guide to Apache Spark and Delta Lake』 Chapter 3 より）。

```
from pyspark.sql.functions import when, coalesce, datediff, current_date, struct, explode, collect_list
# 条件分岐 (SQL の CASE 相当)
df.withColumn("tier", when(col("c_acctbal") > 5000, "gold")
                        .when(col("c_acctbal") > 0, "silver").otherwise("negative"))
# NULL を代替値で埋める
df.withColumn("filled", coalesce(col("maybe_null"), lit("(unknown)")))
# 日付: 経過日数
df.withColumn("days_since", datediff(current_date(), col("o_orderdate")))
# 複雑な型: struct でネスト、collect_list → explode で展開
df.withColumn("profile", struct("c_mktsegment", "c_nation"))
df.groupBy("c_mktsegment").agg(collect_list("c_custkey").alias("keys")).withColumn("k", explode("keys"))
```

6. データソース：読み込み・書き込み

分析用途では **Parquet**（列指向・高速）や **Delta** が定番です。Databricks ではファイル入出力に `/Volumes/...`（Unity Catalog ボリューム）を使います。

```
# 書き込み (mode は overwrite / append / error(既定))
df.write.format("parquet").mode("overwrite").save(f"{volume}/out_parquet")
df.write.format("csv").option("header", True).mode("overwrite").save(f"{volume}/out_csv")
df.write.mode("overwrite").saveAsTable(f"{catalog}.{schema}.result") # Delta テーブル

# 読み込み
spark.read.format("csv").option("header", True).option("inferSchema", True).load(path)
spark.read.format("parquet").load(path) # スキーマは自己記述
spark.table(f"{catalog}.{schema}.result")
```

テーブルとデータの分離：「テーブル＝メタストアの定義」「データ＝ストレージ上のファイル」。RDB と違い、1 つのデータに複数テーブルを紐づけることもできます。

Part 2 — 中級：実践的なデータエンジニアリング

結合・集計・SQL 連携に加え、テスト可能な関数設計、動的処理、パフォーマンスチューニングまで踏み込みます。

7. 結合・集計・メソッドチェーン

```
from pyspark.sql.functions import avg, count, sum as _sum # 組み込み sum と衝突回避

df_joined = df_order.join(df_customer,
    on=df_order["o_custkey"] == df_customer["c_custkey"], how="inner") # inner/left/outer

df_customer.groupBy("c_mktsegment").agg(
    avg("c_acctbal").alias("avg_balance"),
    count("*").alias("n_customers"))

# メソッドチェーンで読みやすく
(df_order.filter(col("o_orderstatus") == "F")
    .groupBy("o_orderpriority")
    .agg(count("o_orderkey").alias("n_orders"))
    .sort(col("n_orders").desc()))
```


8. Spark SQL を PySpark から使う (4 スタイル)

PySpark と SQL は相互運用できます。チームの得意な書き方を選べます。

```
# 1) spark.sql (純粋な SQL)
df.createOrReplaceTempView("v")
spark.sql("SELECT c_mktsegment, COUNT(*) FROM v GROUP BY c_mktsegment")

# 2) selectExpr (SQL 式で選択・変換)
df.selectExpr("c_custkey", "upper(c_mktsegment) AS seg", "CAST(c_acctbal AS int) AS bal")

# 3) expr (withColumn の中で SQL 式)
from pyspark.sql.functions import expr
df.withColumn("band", expr("CASE WHEN c_acctbal>5000 THEN 'high' ELSE 'mid' END"))

# 4) filter / where に SQL 文字列、alias で別名
df.alias("c").filter("c.c_acctbal > 9000").select("c.c_custkey")
```

値の取得 (collect の作法)

重要： `collect()` / `toPandas()` は**全データをドライバーに集める**ため、大量データでは使わず、必ず事前に `filter` / `limit` で絞り込みます。

```
row = df.filter("c_custkey = 1").select("c_name").collect()[0]
row[0]; row["c_name"]; row.c_name          # 位置 / キー / 属性アクセス
names = [r["c_name"] for r in df.limit(5).collect()] # 複数値をリスト化
```

9. 「DataFrame in, DataFrame out」 設計と単体テスト

入力も出力も **DataFrame** にする関数設計は、再利用性とテスト容易性を高める王道パターンです。

```
def cast_int_col_as_int(df):
    """DataFrame in / DataFrame out。Databricks は既定で ANSI モードのため、
    "12.3" のような文字列も丸めるなら double 経由で int にキャストする。"""
    return df.withColumn("int_col", df.int_col.cast("double").cast("int"))

import unittest
class Test(unittest.TestCase):
    def test_cast(self):
        spark = SparkSession.builder.getOrCreate()
        test_df = spark.createDataFrame([{"int_col": "1"}, {"int_col": "12.3"}], "int_col string")
        expected = spark.createDataFrame([{"int_col": 1}, {"int_col": 12}], "int_col integer")
        self.assertEqual(cast_int_col_as_int(test_df).collect(), expected.collect())
```

変数・クラス・Config の活用

```
# 不変な DataFrame を辞書に貯めて処理過程を追跡
stages = {}
stages["raw"] = df
stages["casted"] = cast_int_col_as_int(df)

# 共通処理を staticmethod に、クラス変数を Config として使う
class SparkUtilities:
    is_databricks = True
    @staticmethod
    def show(df, n=5):
        df.display() if SparkUtilities.is_databricks else df.show(n)
```

10. 制御フローによる動的処理

メタデータ（スキーマ）を使い、列を動的に処理する実践テクニック。スキーマ統一や動的 SQL 生成に役立ちます。

```
# (a) スキーマに基づく型の自動変換
target_types = {"id": "int", "amount": "double", "ts": "date"}
tgt = df
for c in df.columns:
    want = target_types.get(c)
    if want == "date":
        tgt = tgt.withColumn(c, F.to_date(F.col(c)))
    elif want:
        tgt = tgt.withColumn(c, F.col(c).cast(want))

# (b) 列名から SELECT 文を動的生成
select_clause = "\n    , ".join(df.columns)
spark.sql(f"SELECT\n    {select_clause}\nFROM my_table LIMIT 3")
```

11. Delta Lake：信頼性の高いテーブル運用

Databricks のテーブルは既定で **Delta Lake** 形式です。Delta Lake はデータレイク上のストレージ層で、**ACID トランザクション**・大規模メタデータ・バッチ/ストリーミング統合・**タイムトラベル**を提供します。変更履歴は `_delta_log` (トランザクションログ) に記録され、MVCC により安全に更新できます (『The Data Engineer's Guide to Apache Spark and Delta Lake』Chapter 4 より)。

ACID トランザクション

同時実行でも一貫性を保証。読み手は更新前のバージョンを参照できる。

スキーマ強制・進化

型不一致の書き込みを拒否。 `mergeSchema` でスキーマ進化も可能。

タイムトラベル

`VERSION AS OF /`
`TIMESTAMP AS OF` で過去の状態を参照・復元。

運用コマンド

`OPTIMIZE` (小ファイル圧縮)・`Z-ORDER`・`VACUUM` (不要ファイル削除)。

```
from delta.tables import DeltaTable
df.write.format("delta").mode("overwrite").saveAsTable(tbl) # 既定で Delta
dt = DeltaTable.forName(spark, tbl)

# 従来は全書き換えが必要だった操作を 1 文で (ACID)
dt.delete("c_acctbal < 0") # DELETE
dt.update("c_mktsegment = 'BUILDING'", {"c_mktsegment": "'CONSTRUCTION'"}) # UPDATE

# MERGE (upsert) : 更新・挿入・重複排除を 1 文で
(dt.alias("t").merge(updates.alias("u"), "t.c_custkey = u.c_custkey")
    .whenMatchedUpdateAll().whenNotMatchedInsertAll().execute())

# 履歴・タイムトラベル・最適化
spark.sql(f"DESCRIBE HISTORY {tbl}")
spark.sql(f"SELECT COUNT(*) FROM {tbl} VERSION AS OF 0")
spark.sql(f"OPTIMIZE {tbl} ZORDER BY (c_custkey)")
```

なぜ便利か： 従来のデータレイクで DELETE/UPDATE はテーブル全体の再作成が必要でした。Delta Lake なら 1 文で済み、しかも履歴が残るため誤操作のロールバックや監査 (GRC) にも

使えます。

12. パフォーマンスチューニング

テクニック	内容
DataFrame / 組み込み関数を優先	JVM 実行で高速。RDD・Python UDF は Python プロセスで遅い
データ量を早く減らす	パイプラインの早い段階で <code>filter</code> / <code>select</code> 。結合は小さいデータ同士を先に
broadcast join	小さいテーブルを <code>broadcast()</code> してシャッフルを回避
cache / persist	繰り返し使う DataFrame をメモリに保持して再計算を防ぐ（汎用クラスター向け。Serverless はキャッシュ自動管理のため明示 cache は非対応）
パーティション調整	<code>repartition</code> / <code>coalesce</code> で並列度を最適化
列指向フォーマット	Parquet / Delta で高速読み込み・列ブローニング
Skew（偏り）に注意	特定パーティションにデータが集中すると処理待ちが発生
explain で確認	<code>df.explain()</code> で実行計画と最適化結果を確認

```
from pyspark.sql.functions import broadcast
df_order.join(broadcast(df_customer), "c_custkey", "inner") # BroadcastHashJoin
df.cache(); df.count() # 実体化してキャッシュ
df.explain(mode="simple") # 実行計画
```

pandas との相互変換と Apache Arrow

集計後の小さな結果は pandas に変換して可視化・後処理できます。**Arrow** を有効にすると変換が高速化されます。

```
spark.conf.set("spark.sql.execution.arrow.pyspark.enabled", "true")
pdf = df.groupBy("c_mktsegment").agg(avg("c_acctbal")).toPandas() # 小さく集計してから
df_back = spark.createDataFrame(pdf) # pandas → Spark
```

13. 注意点・ベストプラクティス総まとめ

カテゴリ	ポイント
遅延評価	アクションが実行されるまで変換は計算されない。デバッグ時はこの違いを意識
大量データ収集	<code>collect</code> / <code>toPandas</code> は全件をドライバーへ。件数を絞ってから使う
ループ回避	<code>for</code> で 1 行ずつ処理せず、列式で一括処理（分散処理に乘せる）
関数の衝突	<code>sum</code> / <code>max</code> は Python 組み込みと衝突。 <code>F.</code> 接頭辞か別名 <code>import</code>
UDF よりビルトイン	Python UDF はシリアライズコスト大。まず組み込み関数・SQL 式で書けないか検討
結合キーの曖昧さ	同名列があると結合後に曖昧参照。事前に <code>withColumnRenamed</code> か条件明示
SparkSession	プロセスごとに独立。並列化ライブラリでの共有は避ける
設計	DataFrame in / DataFrame out で関数化し、単体テスト可能に
Unity Catalog	<code>catalog.schema.table</code> の 3 階層。入出力は <code>/Volumes/...</code> を使う
使いどころ	小規模なら 1 台でサンプリング処理の方が速いことも。大規模＋分散が必要な場面で生きる
ランタイム要件	カスタムデータソース等は DBR 15.4 LTS 以降。 <code>[UNSUPPORTED_FEATURE]</code> 時は確認

14. ハンズオン手順

付属のノートブック `notebooks/01_pyspark_handson.py` を Databricks 上で実行します。Part 1（初級）→ Part 2（中級）の順に手を動かします。

- 1 **Git フォルダの準備**：Databricks で `pyspark_handson` リポジトリを Git フォルダとして追加（または最新へ Pull）。
- 2 **ノートブックを開く**： `notebooks/01_pyspark_handson.py` を開く。
- 3 **コンピューターをアタッチ**：Serverless もしくは DBR 15.4 LTS 以降の汎用クラスター。
- 4 **パラメータ設定**：ウィジェットで出力先カタログ・スキーマ、生成件数を設定。
- 5 **Part 1（初級）**：概念 → SparkSession → 遅延評価 → DataFrame 作成 → 基本操作 → データ型操作 → データソース。
- 6 **Part 2（中級）**：結合・集計 → SQL 連携 → 設計&テスト → 動的処理 → Delta Lake → 性能 → pandas/Arrow → UDF。
- 7 **後片付け（任意）**：最終セルのコメントを外すと作成スキーマを削除。

生成データ：既定で顧客 10,000 件・注文 50,000 件。 `num_customers` / `num_orders` ウィジェットで増減可。

15. 参考リンク

- [Databricks の PySpark \(概要\)](#)
 - [PySpark の基本](#)
 - [PySpark のデータソース](#)
 - [The Data Engineer's Guide to Apache Spark and Delta Lake \(Databricks eBook\)](#)
 - [PySpark開発時に最低限知っておくべき7つの知識 \(manabian, Qiita\)](#)
 - [Pythonで大量データ処理！PySparkを用いたデータ処理と分析のきほん \(Chie Hayashida, SpeakerDeck\)](#)
 - [本ハンズオンのリポジトリ](#)
-